

Passing Parameters to an AWS Lambda Function

Using a Blueprint

To Begin with the Lab

Summary of the Lab

A Python-based **AWS Lambda** function was created using an AWS-provided **Blueprint** instead of authoring the code from scratch. The function automatically used the sample code included in the blueprint and an **IAM** execution role was created during deployment. A custom test event containing JSON key-value pairs was configured and passed to the Lambda function during testing. The function execution was verified by checking both the returned output and the execution logs stored in **Amazon CloudWatch Logs**, confirming that the input parameters were received successfully.

Understanding AWS Lambda Blueprints and Test Events

An **AWS Lambda Blueprint** is a pre-built function template provided by AWS for common serverless use cases, allowing developers to start with working sample code instead of creating everything from scratch. A **Lambda Test Event** is a JSON payload that simulates the data a Lambda function receives during execution, making it easy to test different inputs without connecting real AWS services. Together, blueprints and test events help developers build and validate Lambda functions much faster during development. This approach reduces coding effort, speeds up testing, and simplifies learning serverless applications.

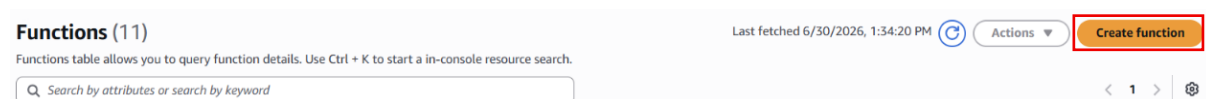
Example:

A Lambda function expects customer information in a JSON event containing `customerId`, `customerName`, and `city`. By creating a test event with these values, the function can be verified immediately, ensuring it correctly processes and returns the expected output before being connected to an actual application.

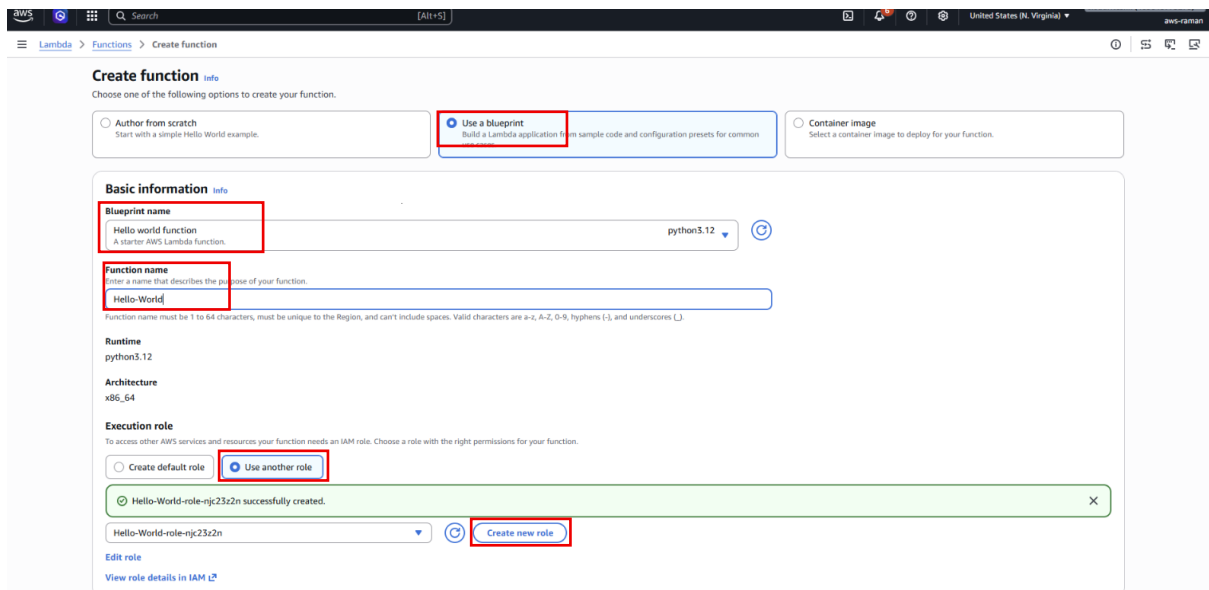
Lab Steps

Step 1: Create a Lambda Function Using a Blueprint

- Open the **AWS Lambda** console.
- Click **Create function**.

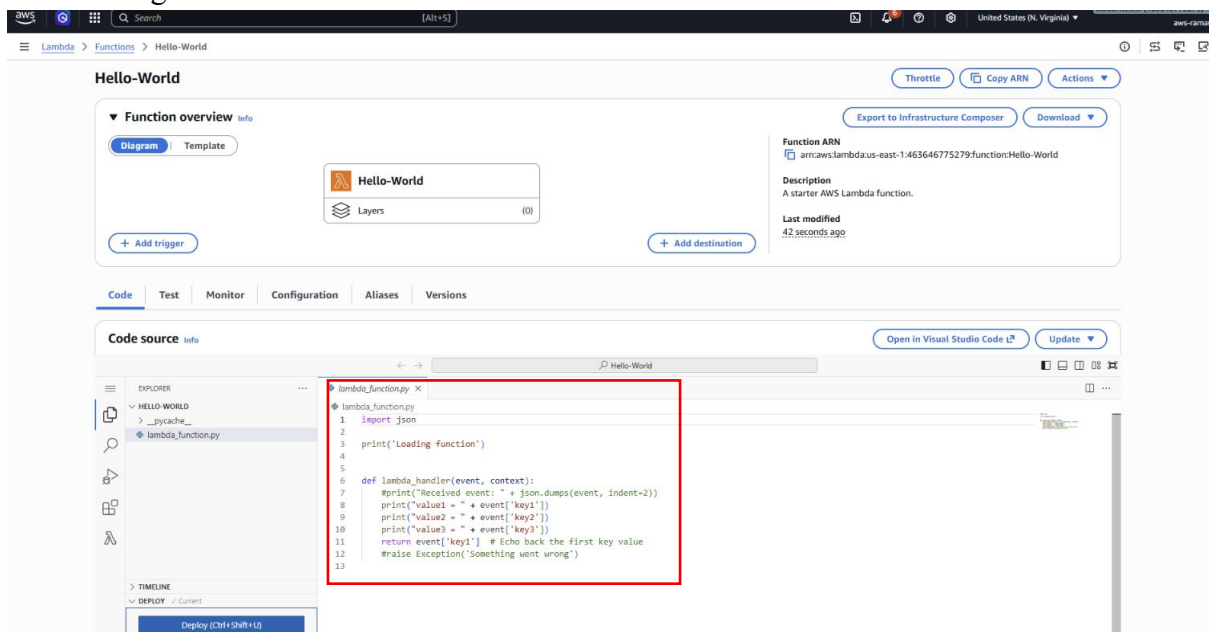


- Select **Use a blueprint** instead of **Author from scratch**.
- Choose the **Hello World** blueprint.
- Select the **Python 3.12** runtime.
- Enter a function name (for example, **Hello-World**).
- Choose **Create a new role with basic Lambda permissions**.
- Click **Create function**.



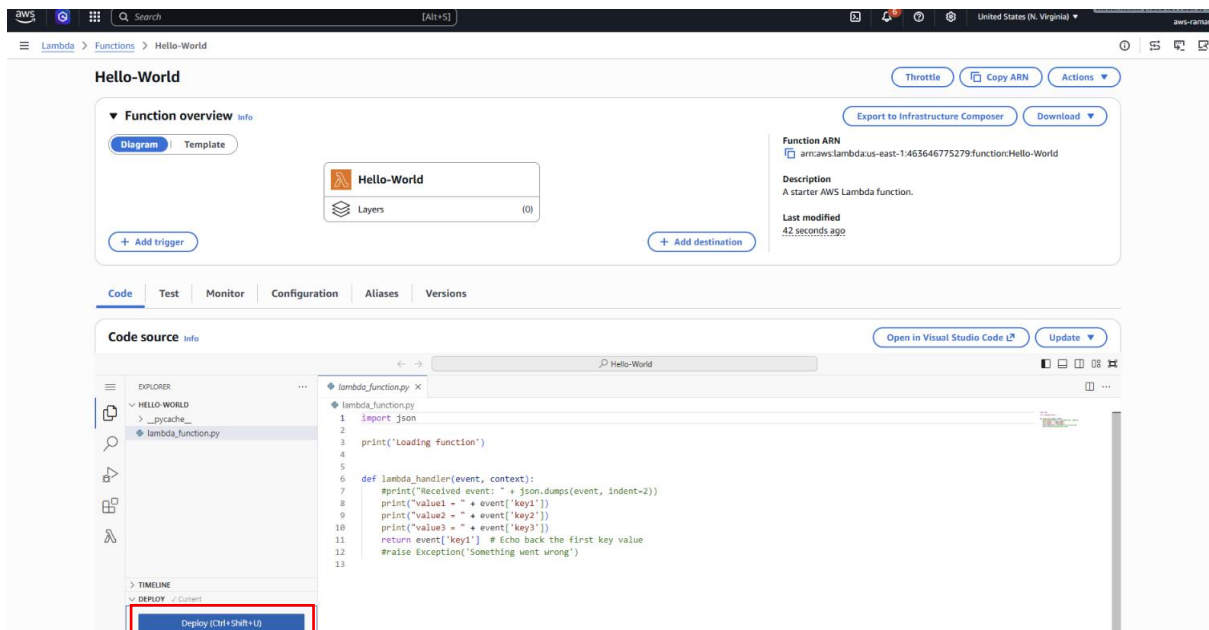
Step 2: Review the Blueprint Code

- Open the generated Lambda function.
- Observe that the blueprint already contains sample code.
- Notice that the function expects input values such as **key1**, **key2**, and **key3** from the test event.
- Understand that blueprints provide ready-made code for common AWS use cases, reducing the need to write code from scratch.



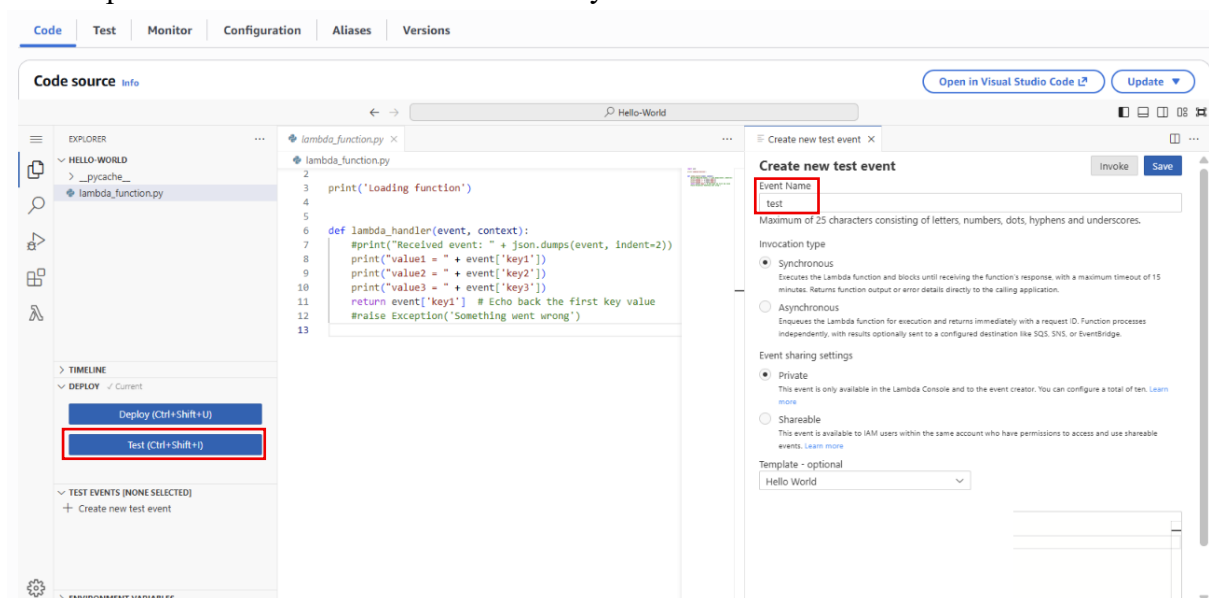
Step 3: Deploy the Lambda Function

- Click **Deploy** to save the Lambda function
- Wait until the deployment completes successfully.



Step 4: Create a Test Event

- Click **Test**.
- Select **Create new test event**.
- Enter a test event name (for example, **test**).
- Keep the default JSON structure or modify it as needed.



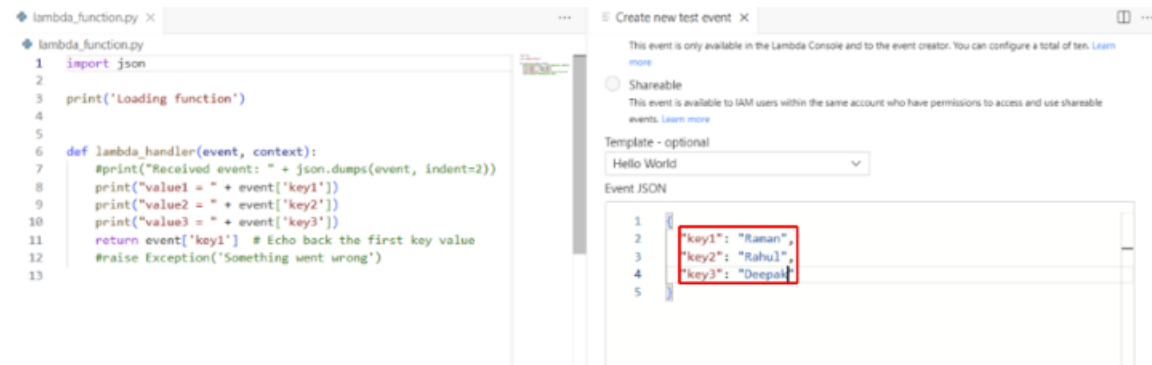
Step 5: Add Input Parameters

- Edit the JSON test event.
- Replace the sample values with your own values.

Example:

```
{
  "key1": "Raman",
  "key2": "Rahul",
  "key3": "Deepak"
}
```

- Click **Save**.

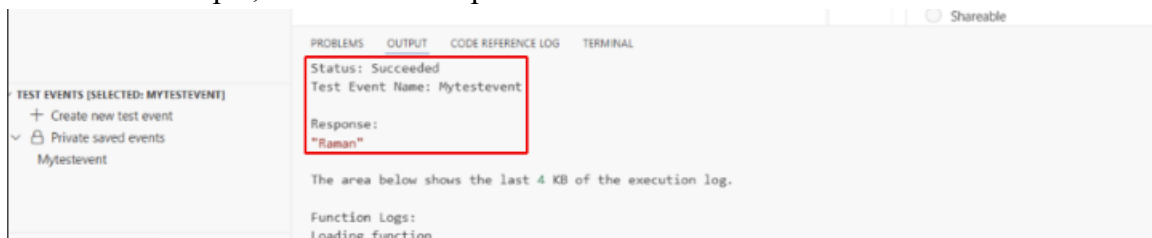


Step 6: Test the Lambda Function

- Click **Test** again.
- Wait for the execution to complete.
- Review the **Execution result**.

Step 7: Verify the Output

- Observe that the function returns the value of **key1**.
- In this example, the returned output is: Cloud Folks Hub



Step 8: Review the Function Logs

- Scroll to the **Execution logs** section.
- Observe that all input parameters (**key1**, **key2**, and **key3**) are printed in the logs.
- These logs are also available in **Amazon CloudWatch Logs**.



Step 9: Understand the Result

- The values entered in the test event are passed to the Lambda function as input.
- The function processes the JSON event.
- The return statement displays the value of **key1** as the output.

- Other parameter values are visible in the execution logs for debugging and verification.

